

Mnist Scratch NN - multilayer perceptron

Mnist dataset:
 $28 \times 28 \Rightarrow 784$ pixel
 row col
 black white
 0-255

MNIST Header (16 bytes)
 group of 4

label viewed in UInt32
 [4] $\rightarrow$ group of 4 bytes: 32 bits

label file: 8 byte header, [4] [4]
 image file: 16 byte header, [4] [4] [4] [4]

black background
 text is white

input: 784

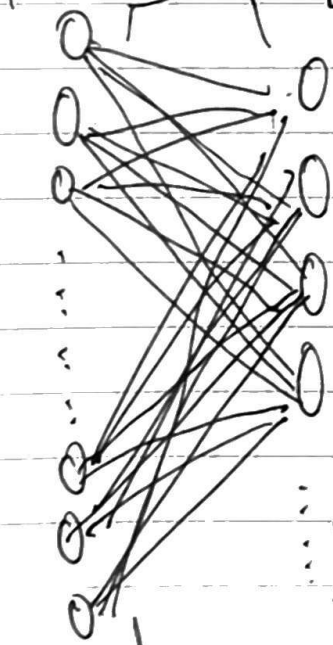
output: 10 (0-9)

label
 [4] $\rightarrow$ 2049: label file
 [4] $\rightarrow$ number of labels
 $\sim 60,000$

image
 [4] $\rightarrow$ 2051: img file
 [4] $\rightarrow$ number of images
 $\sim 60,000$
 [4] $\rightarrow$ num rows (28)
 [4] $\rightarrow$ num cols (28)

Use identifier to double check file

layer a weights b: bias

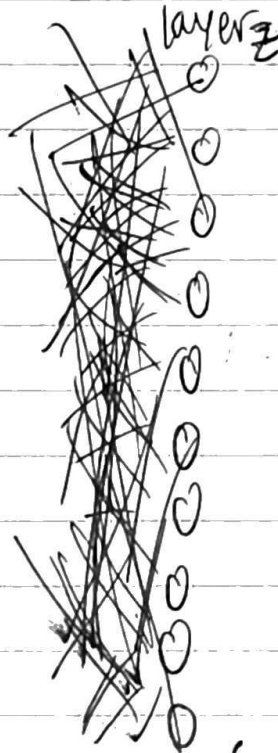


hidden layer

?? layers
 ?? neurons?

each neuron

$\sum a_n w_n + b$
 most params are
 weights $\rightarrow$ each
 neuron has a different
 weight per input



output
 10-9)

First stage
 is forward pass
 NN, no training.

We will use
 2 hidden layer
 with 16 neurons
 in each layer
 so

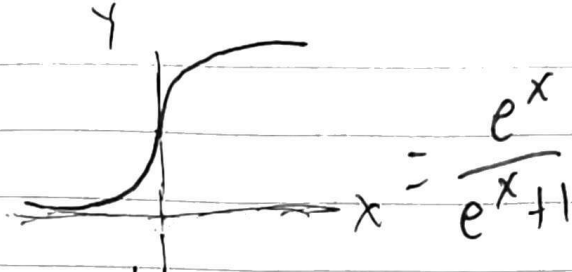
$784 \rightarrow 16 \rightarrow 16 \rightarrow 10$

~~784~~ ~~16~~ ~~16~~ ~~10~~
 $\rightarrow$ parameters = $n \times m + m$
 weight bias

$784 \times 32 +$
 $16 \times 16 +$
 $16 \times 10 +$
 $+ 16 + 16 + 10$ (bias) $=$ 13,002 parameters

Math feed forward

Sigmoid function: $\sigma = \frac{1}{1+e^{-x}}$
logistic curve

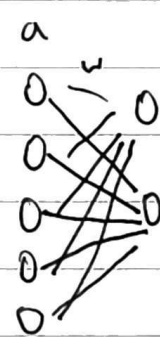


activation function to smoothen inputs.
"normalizes" values to between 0 and 1
well not really

as $z \rightarrow +\infty$ $\sigma(z) \rightarrow 1$ but never reaches
as $z \rightarrow -\infty$ $\sigma(z) \rightarrow 0$ but never reaches
steepest at $y=0.5$
squashes inputs to between 0 and 1
has pros and cons (flattening) but good enough here

next time rewatch video and map out formulas again
for forward pass. require a bit of visualization

input: $a^{(0)}$
 $\hookrightarrow$ activation, layer 0
 $W_1 a_1 + W_2 a_2 + \dots + W_n a_n + b$
put weighted sum into sigmoid
bias for inactivity $= 1$



each circle on left represents the activation input, each line is a weight for each.

$$a_0^{(1)} = \sigma(W_{0,0} a_0^{(0)} + W_{0,1} a_1^{(0)} + \dots + W_{0,n} a_n^{(0)} + b_0)$$

$\hookrightarrow$ matrix multiplication

$$\sigma \left(\begin{bmatrix} W_{0,0} & \dots & W_{0,n} \\ W_{1,0} & \dots & W_{1,n} \\ \vdots & & \vdots \\ W_{k,0} & \dots & W_{k,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{bmatrix} \right)$$

$= a^{(1)}$ (full layer)
 $a^{(1)} = \sigma(W a^{(0)} + b)$

$W_{0,0} \dots W_{0,n}$ represents all weights for each $a^{(0)}$ for one neuron
one neuron



imagine the multiplication:

$$\begin{bmatrix} W_{0,0} & \dots & W_{0,n} \\ \vdots & & \vdots \end{bmatrix} \cdot \begin{bmatrix} a_0^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix}$$

the weights leading to another neuron would be like $\uparrow$ changes

next pg cont.

$$\begin{bmatrix} w_{0,0} & \dots & w_{0,n} \\ w_{1,0} & \dots & w_{1,n} \\ \vdots & & \vdots \\ w_{k,0} & \dots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix}$$

multiplication visualized.

$$\text{let } B = \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix}$$

layer by layer (row) $\downarrow$

$$w_{0,0} \dots w_{0,n}$$

$$\downarrow \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix}$$

$\Rightarrow$

$$\begin{bmatrix} w_{0,0} \cdot a_0^{(0)} \\ + \\ w_{0,1} \cdot a_1^{(0)} \\ + \\ \vdots \\ w_{0,n} \cdot a_n^{(0)} \end{bmatrix}$$

rotate, not swap $\uparrow$

let this weighted sum be S

thus, we can rewrite as

$$a^{(1)} = \sigma(S + B) \quad \text{where}$$

$$\begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ a_2^{(1)} \\ \vdots \\ a_n^{(1)} \end{bmatrix} = a^{(1)}$$

$$= a^{(1)}$$

, S,

$$B = \begin{bmatrix} b_0 \\ \vdots \\ b_n \end{bmatrix}$$

that is

W, B are updated to next layer's values.

$$\begin{aligned} a^{(1)} &= \sigma(Wa^{(0)} + B) \\ a^{(2)} &= \sigma(Wa^{(1)} + B) \\ &\dots \end{aligned}$$

Now to implement in python:

- initialize
- data loader
- compute
- save params (for later)

after implementing can optimize a little:

- batch processing instead of single-length 784 vector.
- relu, cross entropy instead
- can first code compute part with arbitrary nums, not loaded to test.
- modularize.

numpy (math)
mat plot lib (visualization)
pickle (saving)
struct (unpacking data to read)
 $\hookrightarrow$ or with numpy too

initialization (could data load first) $\rightarrow$ can skip data load, weights can be loaded from model after training

- random weights
- small positive bias
- load layer sizes

- create/define layer object
- neurons/activations

how to link to next layer?

need this for weights, bc # weights depends on # activations in next layer or just index? $i=0,1,2,\dots$

- set ~~next~~ ~~2~~ ~~load~~ ~~(prev)~~ ~~our~~ ~~list~~ ~~start~~? or python list?
- get # activations from next to set weights?
- how about biases?

- instead, remove defined # inputs and let neuralnet class handle them?

tuple of layer input, output size

n rows $n \times m$
 m columns

n
16

m 784

where do we

get n and m ?

m is size of input $\rightarrow$

n is size of output

m input

1	...	784
$\vdots$		
16		

define layer object

define NN object

NN creates layers with

sizes as specified

compute

put in data loader?

- receive numpy array of 784 numbers
- the inputs are from data, normalized.
- the data ranges 0-255, we will divide by 255
- so result input is array of 784 nums from 0 to 1.
- np.array() converts list to array we need

L)

init loop from layer sizes

layers in [784, 16, 16, 10] = Layer(784, 16)

layers = []

for i in layer_sizes: Layer(16, 16)

try: add Layer(i, i+1), Layer(16, 10)
to layers

prept:

pass

or just can do

~~for i in range(len(layer_sizes)-1)~~

self.layers = ~~Layer(i, i+1)~~
1. Layer[layer_sizes[i], layer_sizes[i+1]]
forgot to use the nums to index, solves issue

methods: (for now)
forward()

then some for data like:

- layers (get layers)
- get layer size

init:

- take in layer sizes from input
 - create list with pairs of input and output (layers)
- list of layers

to add later:

- backpass
- training, etc.

~~layer~~ Forward pass

loops forward pass until output

$n \times m$

row $\times$ ~~layer~~ column
output $\times$ input
 $\uparrow$ $\uparrow$
 n m

layer class

methods:

forward() sends to next layer

init:

- input size (given by NN object)
- output size

Weights array (matrix) $\rightarrow n \times m$:
2D array $\begin{vmatrix} 1 & \dots & 284 \\ \vdots & & \\ 16 & & \end{vmatrix}$

Bias array (vector) $\rightarrow$ 1D array - 1 per neuron in next
 $\hookrightarrow$ initialize as zeros

add new attributes later

- input
- weighted sum $\rightarrow (W a + b)$
- activation applied to weighted sum (σ)
- output

Neural Network forward() method

loop forward until final output

ex: given ~~layers~~ layers $\rightarrow$ Layer(784, 16)
Layer(16, 16) 3 layer
Layer(16, 10) obs

layers[1].forward(input) $\rightarrow$ return output

layers[2].forward($\swarrow$)

⋮

no more left in layers $\rightarrow$ for i in layers

for i in layers:

~~next = i.forward~~
~~next = i.forward~~
~~next = input~~

input = i.forward(input)

return input

i guess
we don't
need to call
it input...

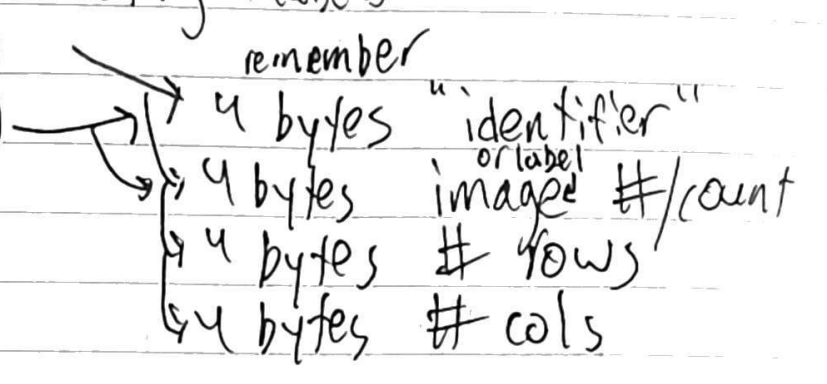
Data Loader

```
import struct
import numpy as np
```

```
read first 16 bytes (img)
read first 8 bytes (label)
```

```
2051 - img
2049 - label
```

files
 training images
 training labels
 testing images
 testing labels



after label/header, read rest of file like normal. byte by byte, which are converted to unsigned 8-bit unsigned int (0-255) and stored in numpy array.

then need to reshape array for images

..... 47,000,000 ->
 much more reasonable
 to convert & matches
 shape of network.

1		784
1		784
1		784
:	:	:
:	:	:
:	:	:

labels can stay flat bc its not 784 nums for each, only a single num.

label
~~1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100~~ 1 1 60,000

Training

You should only see the total nums after each epoch (complete pass of dataset).

it would be too much to print the cost of each image, so we log the ~~avg~~ total avg cost after every epoch.

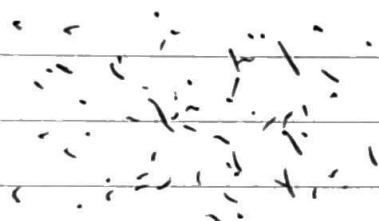
We can then compile this later for further visualization later if we want.

When theres more, we want to see less.

Hide

Show args

compile



.....

.

each image

each epoch

???

next: write cost function/code then we can begin with back propogation.

also need to save params. (separate file... still need random initialization in layer.py)

lost $\rightarrow$ mean squared error

$$\left(\frac{\text{output} - \text{expected}}{\text{output}} \right)^2 + \text{all} \dots = \text{cost}$$

squared so - no negative $\swarrow$ 10 of these

for us, around 10 numbers added to get cost

low cost when accurate
large when bad network.

then we avg cost across the 60,000 images

next we want to minimize the function

using gradient descent - step in steepest direction

can help prevent overshooting by matching step size with slope.

global min might just be memorization and not actually the best

$-\nabla C(\vec{w})$ tells direction and magnitude of nudge to $(\vec{w})$

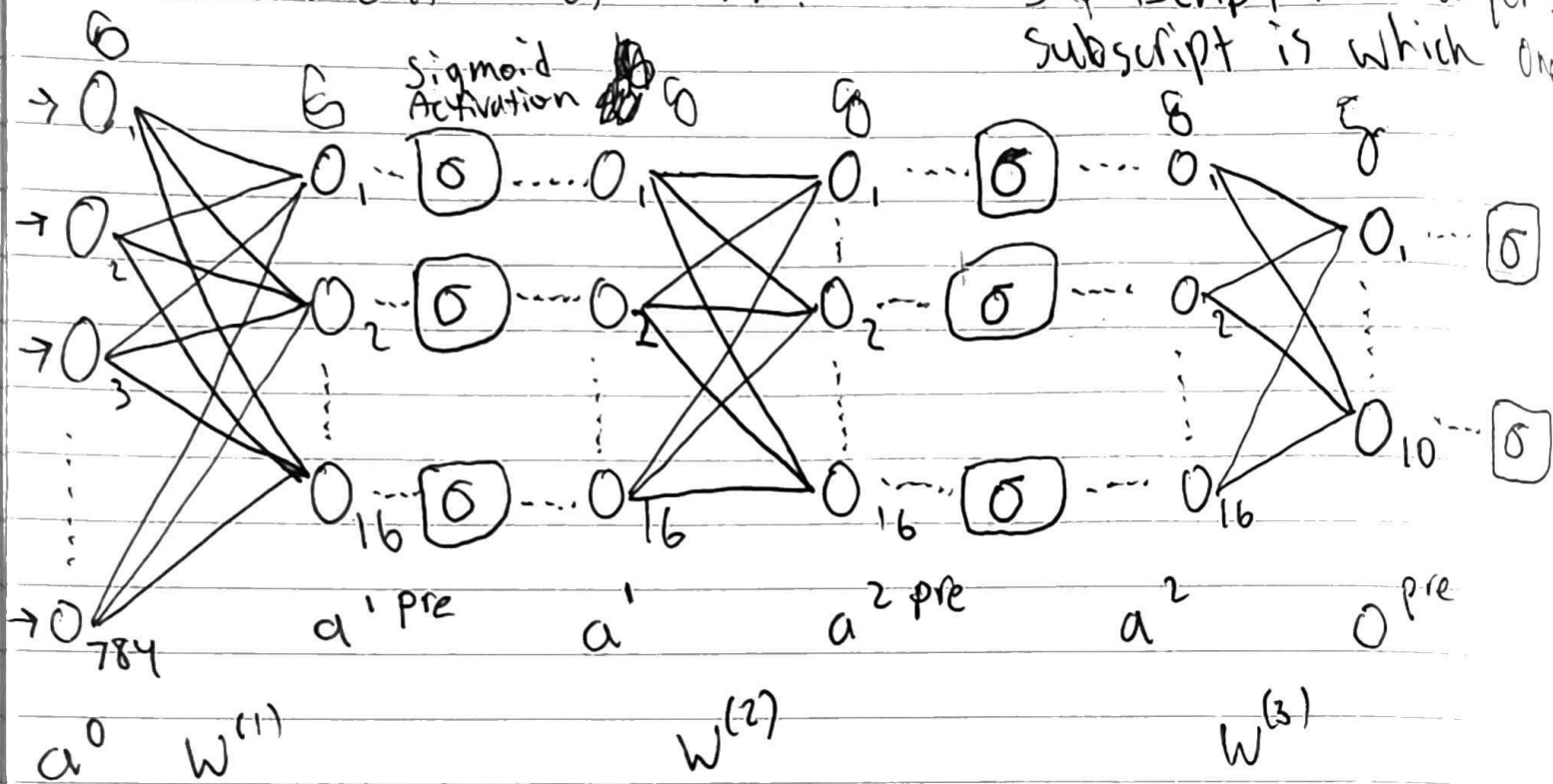
Back propagation next.

Back propagation

to find formula

Our current net work:

Superscript is layer:
Subscript is which one



to back propagate, we must find gradient

$\frac{\partial C}{\partial O}$ and so on backwards.

$$1) C = (O_1^{pred} - O_1)^2 + (O_2^{pred} - O_2)^2 + \dots + (O_{10}^{pred} - O_{10})^2$$

$$\frac{\partial C}{\partial O_1^{pred}} = 2(O_1^{pred} - O_1) \text{ and } \frac{\partial C}{\partial O_2^{pred}} = 2(O_2^{pred} - O_2) \text{ and so on}$$

The partial above is how much a change in O_1^{pred} affects C .

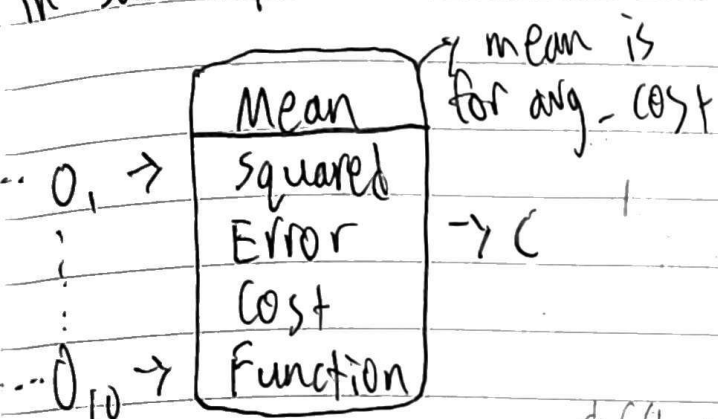
2) now we move backwards using chain rule.

$$\frac{\partial C}{\partial O_1^{pre}} = \underbrace{\frac{\partial C}{\partial O_1^{pred}}}_{\text{already done}} \cdot \frac{\partial O_1^{pred}}{\partial O_1^{pre}} + \underbrace{\frac{\partial C}{\partial O_2^{pred}}}_{\text{already done}} \cdot \frac{\partial O_2^{pred}}{\partial O_1^{pre}}$$

$$\frac{\partial C}{\partial O_1^{pre}} = \sum_{i=1}^{10} \frac{\partial C}{\partial O_i^{pred}} \cdot \frac{\partial O_i^{pred}}{\partial O_1^{pre}}$$

↔
in said layer: ↓

$$\sigma = \frac{1}{1+e^{-x}}$$



$$\frac{d}{dx}((1+e^{-x})^{-1}) = -(1+e^{-x})^{-2} \cdot -e^{-x}$$

$$= \frac{e^{-x}}{(1+e^{-x})^2} = \frac{e^{-x}}{(1+e^{-x})^2}$$

0^{pred}
↳ prediction of actual 0 from our dataset

x is 0₁^{pre}

$$3) \frac{\partial 0_1^{\text{pred}}}{\partial 0_1^{\text{pre}}} = \frac{\partial}{\partial 0_1^{\text{pre}}} \left(\frac{1}{1+e^{-x}} \right) \rightarrow \frac{\partial}{\partial 0_1^{\text{pre}}} \left(\frac{1}{1+e^{-0_1^{\text{pre}}}} \right) = \sigma(x)(1-\sigma(x))$$

$$= \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = \frac{e^{-0_1^{\text{pre}}}}{(1+e^{-0_1^{\text{pre}}})^2} = \boxed{\frac{e^{-0_1^{\text{pre}}}}{1+e^{-0_1^{\text{pre}}}} \cdot \frac{1}{1+e^{-0_1^{\text{pre}}}}}$$

$$\frac{\partial 0_2^{\text{pred}}}{\partial 0_1^{\text{pre}}} = \frac{\partial}{\partial 0_1^{\text{pre}}} \left(\frac{1}{1+e^{-0_2^{\text{pre}}}} \right) = 0 \text{ (no effect)}$$

actually, we don't need to "chain" out this much because 0₂^{pre} doesn't affect 0^{pred} in our sigmoid activation (unlike the softmax in the video we were following. While this will be useful later when we change to softmax on the output, for our scenario: simply

$$\frac{\partial \mathcal{L}}{\partial 0_1^{\text{pre}}} = \frac{\partial \mathcal{L}}{\partial 0_1^{\text{pred}}} \cdot \frac{\partial 0_1^{\text{pred}}}{\partial 0_1^{\text{pre}}} \dots \frac{\partial \mathcal{L}}{\partial 0_{10}^{\text{pre}}} = \frac{\partial \mathcal{L}}{\partial 0_{10}^{\text{pred}}} \cdot \frac{\partial 0_{10}^{\text{pred}}}{\partial 0_{10}^{\text{pre}}} \Rightarrow$$

Back propagation

next:

4) $\frac{\partial L}{\partial a_i^2} \dots \frac{\partial L}{\partial a_{16}^2}$
 go back step for weights and biases
 move back further

$$\frac{\partial L}{\partial a_i^2} = \frac{\partial L}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial a_i^2}$$

→ this is 1

and so

remember to sum

$$\frac{\partial L}{\partial a_k^2} = \sum_{i=1}^{10} \frac{\partial L}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial a_k^2}$$

i and k are indices for different layers

$$\frac{\partial L}{\partial a_k^2} = \sum_{i=1}^{10} \frac{\partial L}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial a_k^2} \cdot \frac{\partial a_k^2}{\partial a_k^2} \cdot \frac{\partial a_k^2}{\partial a_k^2} \cdot \frac{\partial a_k^2}{\partial a_k^2}$$

known

b/c $\frac{\partial a_k^2}{\partial a_k^2} = 1$

weights inside chain

as we go back, each step only has 1 new derivative part
 but we reuse calculations from last step.

go until $\frac{\partial L}{\partial W^{(1)}}$ and use computed weight gradients
 to update weights. used to get gradient vector.

$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial W_{1,1}^{(1)}} \\ \frac{\partial L}{\partial W_{1,2}^{(1)}} \\ \vdots \\ \frac{\partial L}{\partial W_{16,10}^{(1)}} \end{bmatrix}$ think of

as $W_{i,j}$

#neurons on left, #neurons on right

gradient vector eval at param values

→ steepest ascent, take the negative for descent, multiplied by learning rate.

$W_{16,10} \Rightarrow 16 \rightarrow 10$

We need all $\frac{\partial C}{\partial W}$, but it takes $\frac{\partial C}{\partial a}$ to get these too.

$$\frac{\partial C}{\partial W^{(3)}} = \frac{\partial C}{\partial O^{pre}} \cdot \frac{\partial O^{pre}}{\partial W^{(3)}} = \frac{\partial C}{\partial O^{pre}} \otimes a^2$$

→ this is broadcast delta

$$\delta_{O^{pre}} = \frac{\partial C}{\partial O^{pre}} = \sum_{i=1}^10 \frac{\partial C}{\partial O_i^{pre}} \cdot \frac{\partial O_i^{pre}}{\partial O^{pre}} \rightarrow \frac{\partial C}{\partial O_i^{pre}} = 2(O_i^{pre} - O_i)$$

$$\frac{\partial C}{\partial W^{(2)}} = \frac{\partial C}{\partial a^{2pre}} \cdot \frac{\partial a^{2pre}}{\partial W^{(2)}} = \delta_{a^{2pre}} \cdot a_1$$

BUT don't forget the sums for each.

$$\frac{\partial C}{\partial W^{(3)}} = \begin{bmatrix} \frac{\partial C}{\partial W_{1,1}^{(3)}} \\ \vdots \\ \frac{\partial C}{\partial W_{16,10}^{(3)}} \end{bmatrix} \rightarrow \frac{\partial C}{\partial W_{1,1}^{(3)}} = \frac{\partial C}{\partial O_1^{pre}} \cdot \frac{\partial O_1^{pre}}{\partial W_{1,1}^{(3)}} = \frac{\partial C}{\partial O_1^{pre}} \cdot a_1^2$$

$$\rightarrow \frac{\partial C}{\partial W_{2,1}^{(3)}} = \frac{\partial C}{\partial O_1^{pre}} \cdot \frac{\partial O_1^{pre}}{\partial W_{2,1}^{(3)}}$$

$$\text{but } \rightarrow \frac{\partial C}{\partial W_{1,10}^{(3)}} = \frac{\partial C}{\partial O_{10}^{pre}} \cdot \frac{\partial O_{10}^{pre}}{\partial W_{1,10}^{(3)}}$$

$$\begin{bmatrix} W_{1,1}^{(3)} & \dots & W_{1,10}^{(3)} \\ \vdots & & \vdots \\ W_{16,1}^{(3)} & \dots & W_{16,10}^{(3)} \end{bmatrix}$$

$$= W_{16,10}^{(3)}$$

$$\begin{matrix} (10) & (10) \\ \delta & a \rightarrow 20 \\ \vdots & \vdots \end{matrix}$$

→ this can write as

$$\frac{\partial C}{\partial W^{(3)}} = \delta_{O^{pre}} \otimes a^2$$

outer product does this for us

$$\delta \otimes a \rightarrow \begin{bmatrix} \delta_1 a_1 & \dots & \delta_1 a_{10} \\ \vdots & & \vdots \\ \delta_{16} a_1 & \dots & \delta_{16} a_{10} \end{bmatrix}$$

Network back ward

Layer back ward

Train train



prev $\rightarrow$ this $\rightarrow$ next
even when going back
we will use this order

Layer back pass

prev $\leftarrow$ this $\leftarrow$ next
left side right side

Change in cost for change in $\text{--- (layer)}: \frac{\partial C}{\partial ?}$

is $y_{\text{this}} \odot a_{\text{prev}}$ $y_{\text{this}} = |y_{\text{next}}| |w^{\text{next}}|$ layer product layers weighted sum sigmoid deriv

aka $y_{\text{this}} = (y_{\text{next}} \odot w^{\text{next}}) \cdot \text{sigmoid_deriv}(w_{\text{sum}})$

If no y_{next} (output), calculate from cost
bias gradient of layer is just δ_{this}
return delta and weights for next

now we need to back prop through the whole network.

- Reverse list of layers bc we go backward.
- input to layer back is y_{next} and w^{next}
- continue until we got gradients for whole network.

later avg gradients across epoch and then adjust. can later do in batches so more updt per epoch.

y_{next} for output is $\frac{\partial C}{\partial o_{\text{pre}}} = 2(\text{output} - \text{actual})$

sigmoid - deriv (elf. layers $[1, 2, \dots, n]$) $\rightarrow$ ws of last layer

reversed layers: o, a^2, a^1, a^0

train
for epoch # times:
- loop through each image

· input $\rightarrow$ output
· network. back (output, actual)

- total dcdB
- total dcdW

avg totals after
print avg-cost for epoch

also load and save params

can do in patches to speed up.
also add saving total epochs. $\downarrow$
or 62,000 updates
instead of 1 each epoch

next also add testing and visualization so we
can try out the trained network.

$\sim 90.5\%$ accuracy after 30-40 epochs.

can: · batch instead of update for each
· move neurons
· change learning rate 0.05 $\rightarrow$ 0.01

but we don't want to overfit